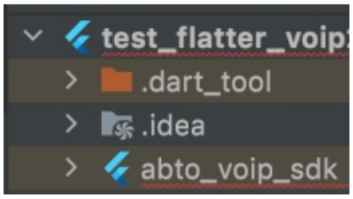
# ABTO VoIP Flutter SDK

## 1. How to setup

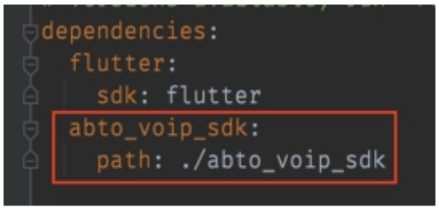
- Put **abto_voip_sdk** folder to your project




```
[YOUR_PROJECT]/abto_voip_sdk
```

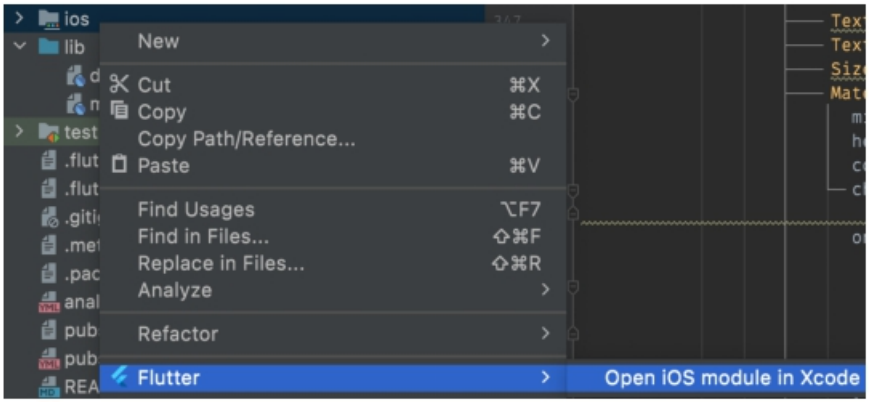
> You can find **example sample** inside of plugin:
> **abto_voip_sdk/example**

- Add lib to dependencies in **pubspec.yaml**

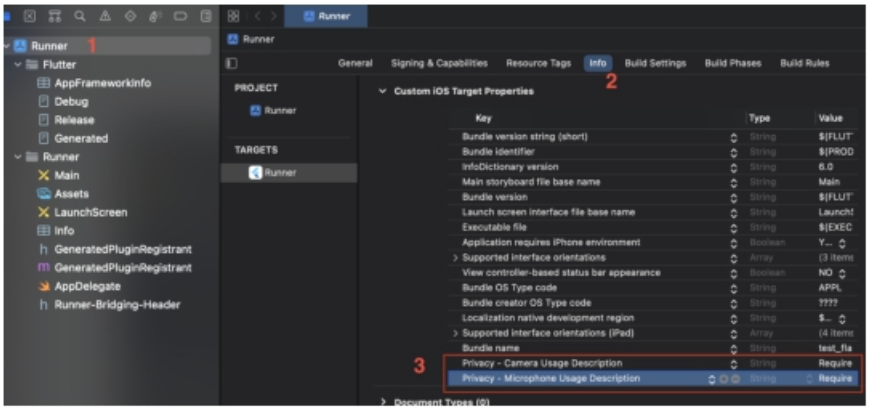



```
abto_voip_sdk:
   path: ./abto_voip_sdk
```

- Setup iOS target
  - Open iOS part as Xcode project




  - Set privacy settings




```
Privacy - Camera Usage Description: Required for video calls
Privacy - Microphone Usage Description: Required for audio calls
```

- Setup Android target
  Flutter Android target is based on ABTO VoIP SIP SDK for Android that requires Android application class implements AbtoApplicationInterface interface. Our Flutter SDK can use 2 standard classes AbtoApplication or AbtoNotificationApplication provided by our native Android SDK or custom Application class from ABTO Flutter SDK:
  1) AbtoApplication - which is a simple SDK class that is injected by default by our SDK module
  2) AbtoFlutterNotificationApplication - which is advanced application implementation that handles background calls even on a locked screen and can be used as a reference implementation of custom application classes.

  If you need custom Application class other than AbtoApplication use replace and name attributes of application tag in AndroidManifest.xml class to override:

```
<application
tools:replace="android:name"
android:name="your custom application class"
...
```

> If you need custom Application class and there is possibility to derive from one of our SDK classes, please extend from one of them, otherwise use code of `org.abtollc.voip.abto_voip_sdk.AbtoFlutterNotificationApplication` class as a reference on how to derive from 3rd party Application class and implement AbtoNotificationInterface .

- Initialise wrapper

```
SipWrapper.wrapper.init();
```

- Set license **userId** and **key** for each platform

```
if (Platform.isAndroid) {
    SipWrapper.wrapper.setLicense(
          "{YOUR_ANDROID_USER_ID}",
          "{YOUR_ANDROID_LICENSE_KEY}"
    );
} else if (Platform.isIOS) {
    SipWrapper.wrapper.setLicense(
         "{YOUR_IOS_USER_ID}",
         "{YOUR_IOS_LICENSE_KEY}"
    );
}
```

```
Trial license prefix example:
"{Trial...}"
"{V0exUTjAafwV...}"

Commercial license prefix example:
"{Licensed_...}"
"{P2FDm7A80...}"
```

> You can get trial keys here https://voipsipsdk.com/product/voip-sip-sdk?attribute_pa_platform=multi_mobile&attribute_pa_language=flutter-dart&attribute_pa_support=support-variation-4191

# 2. How to use

- Add **sip_wrapper** import to use base methods

```
import 'package:abto_voip_sdk/sip_wrapper.dart';
import  'package:abto_voip_sdk/abto_phone_cfg.dart';
```

- Initialise wrapper

```
SipWrapper.wrapper.init();
```

- Setup license keys

```
if (Platform.isAndroid) {
    SipWrapper.wrapper.setLicense(
         androidLicenseUserId,
         androidLicenseKey
    );
} else if (Platform.isIOS) {
    SipWrapper.wrapper.setLicense(
         iosLicenseUserId,
         iosLicenseKey
    );
}
```

- Setup configs

```
// Get configs
final AbtoPhoneCfg configs = await SipWrapper.wrapper.getConfigs();

// Read/Set values
configs.isSTUNEnabled = true;
configs.stunServer = "your server”;
configs.sipPort = 1234;

configs.signalingTransport = AbtoPhoneCfg.SIGNALING_TRANSPORT_UDP;
configs.isICEEnabled = true;
configs.keepAliveInterval = 30; // sec.
configs.inviteTimeout = 3000; // ms.
configs.hangupTimeout = 3000; // ms.
configs.registerTimeout = 15000; // ms.
configs.isUseSRTP = false;
configs.isEnabledAutoSendRtpVideo = true;

// Setup priority for codecs from 0 to 127
configs.videoCodecs[VideoCodec.H264] = 100;
configs.audioCodecs[AudioCodec.G722] = 121;

// Available codecs for both platforms:
// enum AudioCodec { GSM, PCMA, PCMU, ILBC, SPEEDX, G729, G722, G722_1,
AMR, G723, SILK, OPUS }
 // enum VideoCodec { H264, H263, VP8 }

// Update configs
SipWrapper.wrapper.setConfigs(configs);
```

- Work with account registering

```
SipWrapper.wrapper.register(domain, proxy, login, pass, authId,
                            displName, expire);
SipWrapper.wrapper.unregister();
SipWrapper.wrapper.registerListener = RegisterListener(
   onRegistered: () {  },
   onRegistrationFailed: () {  },
   onUnregistered: () {  }
);
```

- Work with call start/end

```
SipWrapper.wrapper.callListener = CallListener(
      callConnected: (number) {  },
      callDisconnected: () {  }
);
SipWrapper.wrapper.incomingCallListener = IncomingCallListener(
   onIncomingCall: (number, isVideoCall) {  }
);

SipWrapper.wrapper.startCall(number, isVideoCall)
SipWrapper.wrapper.pickUpCall(isVideo);
SipWrapper.wrapper.endCall(); // do hangUp or reject depends if call connected or not
SipWrapper.wrapper.hangUpCall(status); // var status = 406;
SipWrapper.wrapper.rejectCall();
```

- Options during the call

```
SipWrapper.wrapper.holdStateListener = HoldStateListener(
   onHoldState: (state) {
   switch(state) {
      case HoldState.ACTIVE: break;
      case
      HoldState.LOCAL_HOLD:
      break; case
      HoldState.REMOTE_HOLD:
      break;
   }
});
SipWrapper.wrapper.hold();
SipWrapper.wrapper.enableSpeaker(true);
SipWrapper.wrapper.mute(true);
SipWrapper.wrapper.startRecord();
SipWrapper.wrapper.stopRecord();
SipWrapper.wrapper.transferCall(number);
SipWrapper.wrapper.setSendingRtpVideo(true);
SipWrapper.wrapper.muteLocalVideo(true);
```

- Work with text messaging

```
SipWrapper.wrapper.textMessageListener = TextMessageListener(
   onTextMessageReceived: ( from, to, message) {
      debugPrint("new message: " + message);
   },
   onTextMessageStatus: (address, reason, success) {
      debugPrint("sent to : " + address + " reason: " + reason);
   }
);

SipWrapper.wrapper.sendTextMessage(number, "Hello");
```

- Use video views

```
import 'package:abto_voip_sdk/abto_video_widget.dart';

SizedBox(
    width: 100,
    height: 120,
    child: VoipVideoWidget(true) // true - for outcoming video
),
SizedBox(
    width: 100,
    height: 120,
    child: VoipVideoWidget(false) // false - for incoming video
)
```

- Work with DTMF

```
SipWrapper.wrapper.sendDtmf(keyCode); // var keyCode = '1'.codeUnitAt(0);
SipWrapper.wrapper.dtmfListener = DtmfStateListener(
onDtmfReceived: (tone) {
  debugPrint("new tone: " + tone);
});
```

- Setup write logs options

```
int logLevel = 5; // from 0 to 5
bool writeToFile = true;
SipWrapper.wrapper.setLogLevel(logLevel, writeToFile);
```

- Get SDK version

```
SipWrapper.wrapper.getVersion()
```